

OAT 1 – Mecânica BELÉM

Autor: Érica Araujo

Mickael Cedraz Alencar

Monyc Luisa Almeida de Cerqueira

Nalbert de Souza Santana

Pedro César Paixão de Jesus

Agenda

O objetivo dessa apresentação é demonstrar a compreensão dos dados temporais da Mecânica e apresentar o desenvolvimento do modelo baseline para a previsão diária de trocas de óleo.

1. Contexto do Projeto:

Contexto do Projeto

2. Inspeção:

Importação, tratamento e indexação temporal

3. Decomposição:

Tendência, sazonalidade de 7 dias e ruído

4. Pipeline:

Engenharia de Lags e Treino Sklearn

5. Previsão:

Avaliação do MAE e previsão do próximo dia

6. Conclusão:

Resultado e Conclusões

Contexto da atividade



O Problema – Gargalo Operacional

- Falta de previsibilidade na manutenção preventiva causa perda de clientes por desabastecimento de óleo/filtros e escala ineficiente de mecânicos.

Impacto no Estoque e Escala $\text{Estoque_Falta} = \text{Picos_Demanda} \times \text{Ociosidade} = \text{Dias_Sem_Previsao}$

A Solução Baseline

- Construção de uma base temporal sólida, tratamento rigoroso de missing values e motor de regressão preditiva com validação cruzada cronológica.
- *# Fluxo Preditivo Mínimo* DataClean -> Decomposição -> RidgePipeline

Inspeção

Estrutura do Dataset

Conversão da coluna **Data** para datetime64 e definição como **DateTimeIndex** cronológico ordenado.

- Garantia da integridade da linha do tempo.
- Mapeamento da série temporal e tipo do índice.

```
df["Data"] = pd.to_datetime(
    df["Data"])
```

```
df = df.set_index("Data").sort_index()
```

```
# Converte Data para datetime
df["Data"] = pd.to_datetime(
    df["Data"],
    format="%Y-%m-%d",
    errors="raise"
)

# Define Data como índice e organiza cronologicamente
df = df.set_index("Data")
df = df.sort_index()

print("Indexação temporal concluída!")
display(df.head())
```

Python

Nomes das Colunas

- Limpeza de espaços em branco nos cabeçalhos para evitar falhas de indexação.

```
df.columns = df.columns.str.strip()
```

Interpolação Temporal

- Preenchimento de dias sem registro com interpolação linear seguida de forward/backward fill.

```
s.interpolate(method="time")
```

Resumo Estatístico

- Identificação de nulos, min/max de datas e verificação de amplitude dos valores de trocas de óleo.

```
df.describe()
```

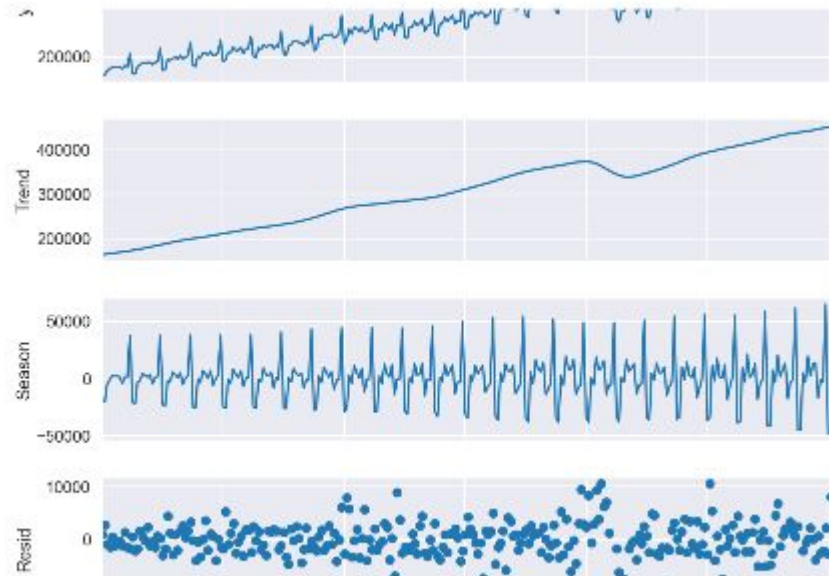
Decomposição Temporal

Separação de Componentes – statsmodels.tsa.seasonal

Adoção do **Modelo Aditivo** para isolar o comportamento de longo prazo e picos cíclicos:

- **Tendência:** Crescimento do fluxo de clientes da oficina.
- **Sazonalidade:** Variação regular com período fixo.
- **Resíduo/Ruído:** Eventos aleatórios e não estruturados.

```
resultado = seasonal_decompose( serie_oleo,  
                                model="additive", period=7 )
```



Janela Sazonal e Média Móvel



- 7 Dias no Período Sazonal

Justificativa do Período Escolhido

- O setor automotivo responde fortemente ao ritmo da semana útil (segunda a sábado vs. domingo fechado/reduzido).
- A média móvel centrada de 7 dias atenua o ruído diário e expõe a real trajetória de demanda.
- *# Média móvel centrada de 7 dias* `media_movel = serie_oleo.rolling(window=7, center=True).mean()`
-

Atributos Temporais (Lags)

Construção de variáveis com base unicamente no histórico passado para impedir o vazamento de dados futuros:

- Trocas_Oleo_Ontem (Lag 1)
- Trocas_Oleo_7_Dias_Atras (Lag 7)
- Media_Ultimos_7_Dias (Janela deslizando)
- Dia_Da_Semana (Categorização temporal)

Divisão Cronológica (80 / 20)

Divisão sequencial estrita mantendo a ordem do tempo sem embaralhar os registros:

```
idx_corte = int(len(base) * 0.80) X_train = X.iloc[:idx_corte] X_test = X.iloc[idx_corte:]
```

Garantia de avaliação realista em dados não vistos.

Pipeline Sklearn & Avaliação



Pipeline de aprendizado de máquina integrado contendo imputação, padronização e o modelo **Ridge Regression**.

- **Simplicidade e Escala** - StandardScaler + SimpleImputer
- **Modelo Regularizado** - Ridge Regression ($\alpha = 1.0$)
- **Métrica MAE (Erro Médio)** - Baixo Erro Absoluto Médio

```
pipeline = Pipeline(steps=[ ("imputer", SimpleImputer(strategy="median")), ("scaler", StandardScaler()),  
("modelo", Ridge(alpha=1.0)) ]) pipeline.fit(X_train, y_train)
```

Considerações finais



- A organização por datas foi essencial pra não bagunçar os dias passados e evitar que o modelo "olhasse para o futuro" antes da hora.
- A decomposição mostrou bem claro que a oficina tem um ritmo que se repete a cada 7 dias (dias de pico e dias mais calmos).
- Conseguimos criar um fluxo de código limpo, rápido e que já deixa tudo pronto desde a limpeza até a previsão final.

Referências



- LET'S DATA. ***Séries temporais com Python***. YouTube, 10 set. 2021. O vídeo explica conceitos de séries temporais e aplicações com Python.
- OLIVEIRA, Rogério de; ALBARRACIN, Orlando Yesid Esparza; SILVA, Gustavo Rocha da. ***Introdução às séries temporais: uma abordagem prática em Python***. São Paulo: Editora Mackenzie, 2024.
- ALURA. ***Séries temporais: detectando anomalias e realizando previsões***. Alura.